

핵심 개념

Claude Code가 프롬프트 캐싱을 사용하는 방법

📄 페이지 복사



Claude Code는 프롬프트 캐싱을 자동으로 관리합니다. 모델 전환이 느린 캐시되지 않은 토큰을 트리거하는 이유, `/compact` 의 비용, CLAUDE.md 편집이 세션 중에 적용되지 않는 이유, 캐시 히트율을 확인하는 방법을 알아봅니다.

프롬프트 캐싱은 Claude Code를 더 빠르고 비용 효율적으로 만듭니다. 캐싱이 없으면 API는 매 토큰마다 전체 기록을 다시 처리합니다. 캐싱을 사용하면 이미 처리한 내용을 재사용하고 변경된 내용에 대해서만 새로운 작업을 수행합니다.

Claude Code는 캐싱을 비활성화하지 않는 한 프롬프트 캐싱을 자동으로 처리합니다. 일부 작업이 캐시를 무효화하고 다음 응답을 더 느리고 비싸게 만들기 때문에 프롬프트 캐싱이 어떻게 작동하는지 아는 것이 여전히 유용합니다. 이 페이지에서는 어떤 작업이 그러한지, 일부 설정이 재시작을 기다리는 이유, 사용량이 높아 보일 때 캐시 성능을 확인하는 방법을 다룹니다.

캐시가 구성되는 방식

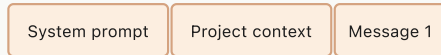
Claude Code에서 메시지를 보낼 때마다 새로운 API 요청을 만듭니다. 모델은 요청 간에 아무것도 기억하지 않으므로 Claude Code는 전체 컨텍스트를 다시 보냅니다. 시스템 프롬프트, 프로젝트 컨텍스트, 모든 이전 메시지 및 도구 결과, 그리고 새로운 메시지입니다. 새로운 콘텐츠는 끝에 추가되므로 각 요청의 대부분은 이전 요청과 동일합니다. Prompt caching은 API가 변경되지 않은 부분을 다시 처리하지 않도록 하는 방법입니다.

API는 각 요청의 시작 부분(프리픽스라고 함)을 최근에 처리한 콘텐츠와 일치시켜 캐시합니다. 일반적인 토큰에서 프리픽스는 전체 이전 요청이고 최신 교환만 새로운 것입니다. 일치는 정확하므로 프리픽스의 어디든지 변경되면 그 이후의 모든 것을 다시 계산합니다. 파일별 또는 세그먼트별 캐싱은 없습니다. API 참조에서 **prompt caching**이 작동하는 방식을 참조하여 기본 메커니즘을 확인하세요.

프롬프트 캐싱이 작동하는 방식 깊은 탐구 필요

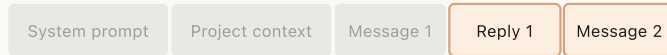
Each turn resends the full context, and the unchanged prefix reads from cache

Turn 1



First turn: no cache yet, the whole request is processed and written

Turn 2



Read from cache (entire turn 1 request)

New: written to cache

Turn 3



Read from cache (entire turn 2 request)

New

Turn 4: the system prompt changed



Changed

Reprocessed: the prefix no longer matches, so even unchanged content is computed and written again

☐ Cached: read from a previous request ☐ New or recomputed: processed and written to cache

프리픽스 일치를 **최대한 활용**하기 위해 Claude Code는 각 요청을 **정렬**하여 턴 간에 거의 변경되지 않는 콘텐츠를 가 먼저 오도록 합니다: **모 어떻게 정렬한다는건지? 무슨 기준을 가지고?**

계층	콘텐츠	변경 시기
시스템 프롬프트	핵심 지침, 도구 정의, 출력 스타일	로드된 도구 정의의 집합이 변경되거나 Claude Code가 업그레이드됨
프로젝트 컨텍스트	CLAUDE.md, 자동 메모리, 범위 지정되지 않은 규칙	세션 시작, 또는 /clear 또는 /compact 후
대화	사용자 메시지, Claude의 응답, 도구 결과	매 턴

대화 계층의 변경은 시스템 프롬프트와 프로젝트 컨텍스트를 캐시된 상태로 유지합니다. 시스템 프롬프트의 변경은 **모든 것을 무효화**합니다. 왜냐하면 모든 이후 콘텐츠가 이제 다른 프리픽스 뒤에 있기 때문입니다. 세 번째 열은 완전한 목록이 아닌 일반적인 트리거를 제공하며, 아래 섹션에서는 세션 시작 시 고정되는 출력 스타일과 같은 콘텐츠를 포함한 전체 집합을 다룹니다.

프리픽스 일치 규칙은 이 페이지의 대부분의 동작을 설명합니다. 예를 들어 **Plan Mode**와 **스킬 로딩**은 대화 메시지로 지침을 추가하므로 **캐시된 프리픽스는 그대로 유지**됩니다.

두 가지 설정은 프롬프트 텍스트의 일부가 아니므로 계층 표에 나타나지 않습니다. 하지만 둘 다 캐시 키의 일부입니다:

- **모델:** 각 모델은 자체 캐시를 가집니다. 모델을 전환하면 콘텐츠가 동일한 경우에도 전체 요청을 다시 계산합니다. 아래의 모델 전환을 참조하세요.
- **노력 수준:** 각 노력 수준은 동일한 모델에 대해 자체 캐시를 가집니다. 세션 중에 변경하면 전체 요청을 다시 계산하며, Claude Code는 변경을 적용하기 전에 확인을 요청합니다. 아래의 노력 수준 변경을 참조하세요.

💡 세션 시작 시 모델과 노력 수준을 선택한 다음 작업 간의 자연스러운 휴식을 위해 `/compact` 를 저장합니다. 작업 중에 변경할수록 캐시 히트율이 높아집니다.

어떻게 테스트를 해볼 수 있을까?

캐시가 있는 위치

캐싱은 서버 측에서 발생하며, 모델을 제공하는 인프라에서 발생합니다. 그 위치는 인증 방식에 따라 다릅니다:

- **API 키, Claude 구독 또는 Claude Platform on AWS:** 캐시는 Anthropic의 인프라에 있으며 Claude API를 통해 액세스됩니다.
- **Amazon Bedrock 또는 Google Cloud의 Agent Platform:** 캐시는 클라우드 제공자의 제공 인프라에 있습니다.
- **Microsoft Foundry:** 요청은 Anthropic의 인프라로 라우팅됩니다.
- **사용자 정의 ANTHROPIC_BASE_URL 또는 LLM gateway:** 캐시는 요청이 전달되는 위치에 있으며, 캐싱이 작동하는지 여부는 게이트웨이에 따라 다릅니다.

각 제공자가 저장하고 처리하는 내용은 데이터 사용을 참조하세요. 캐시가 어디에 있는 항목은 비활성 기간 후에 만료되며, 아래의 캐시 수명에서 TTL과 연장 방법을 다룹니다.

캐시를 무효화하는 작업

이러한 작업으로 인해 다음 요청이 캐시의 일부 또는 전부를 놓칩니다. 한 번의 느리고 비싼 턴을 본 후 새로운 프리픽스가 캐시됩니다. 대부분은 작업 중에 비용이 있다는 것을 알면 피할 수 있습니다. 모델 전환은 뒤따르는 느린 턴을 알아차릴 때까지 무료로 느껴질 수 있습니다.

- 모델 전환
- 노력 수준 변경
- 빠른 모드 켜기
- MCP 서버 연결 또는 연결 해제
- 플러그인 활성화 또는 비활성화
- 전체 도구 거부
- 대화 압축

- Claude Code 업그레이드

모델 전환

각 모델은 자체 캐시를 가집니다. `/model` 로 전환하면 콘텐츠가 동일한 경우에도 다음 요청이 캐시 히트 없이 전체 대화 기록을 읽습니다.

`opusplan` 모델 설정은 Plan 모드 중에 Opus로, 실행 중에 Sonnet으로 확인되므로 각 Plan 모드 토글은 모델 전환이고 새로운 캐시를 시작합니다.

Table 5의 자동 모델 폴백도 모델 전환입니다. 안전 분류기가 요청에 플래그를 지정하면 Claude Code는 기본 Opus 모델에서 다시 실행하고 세션이 계속됩니다.

노력 수준 변경

캐시는 **노력 수준**뿐만 아니라 모델로도 키가 지정되므로 `/effort` 로 전환하면 다음 요청이 캐시 히트 없이 전체 대화 기록을 읽습니다. 대화가 시작되면 Claude Code는 캐시를 무효화할 노력 변경을 적용하기 전에 확인 대화를 표시합니다. 모델의 기본값을 명시적으로 설정하는 것과 같이 이미 적용 중인 동일한 수준으로 확인되는 변경은 대화를 건너뛰고 캐시를 유지합니다.

빠른 모드 켜기

빠른 모드를 활성화하면 캐시 키의 일부인 요청 헤더가 추가되므로 다음 요청이 캐시 히트 없이 전체 대화 기록을 읽습니다. 캐시되지 않은 입력 토큰은 **빠른 모드 요금**으로 청구되므로 세션 시작 시 켜는 것이 긴 세션 깊숙이 켜는 것보다 비용이 적게 듭니다. 비 Opus 모델에서 빠른 모드를 활성화하면 **모델도 전환**되므로 자체적으로 새로운 캐시를 시작합니다.

비용은 대화당 한 번 적용됩니다. 첫 번째 빠른 모드 턴 후 Claude Code는 계속 헤더를 보내고 캐시 키의 일부가 아닌 요청의 속도 설정만 변경합니다. **빠른 모드를 끄기, 속도 제한 후 표준 속도로 자동 폴백, 나중에 다시 켜는 모두 캐시를 유지**합니다. `/clear` 와 `/compact` 는 어차피 그 지점에서 캐시를 다시 구축하므로 이를 재설정합니다.

MCP 서버 연결 또는 연결 해제

도구 정의는 시스템 프롬프트 계층에 있으므로 요청 간에 도구 정의의 집합이 변경되면 캐시가 무효화됩니다.

이게 뭐임? **advisor** 도구를 토글하는 것은 예외입니다: 해당 정의는 캐시 중단점 이후에 있으므로 `/advisor` 를 활성화 또는 비활성화하면 캐시된 프리픽스가 그대로 유지됩니다. **MCP 서버 변경이 이를 수행하는지** 여부는 해당 도구가 **도구 검색**으로 연기되는지 또는 프리픽스에 로드되는지에 따라 달라집니다:

- 연기된 도구, 지원되는 모델의 기본값: 서버 연결, 연결 해제 또는 도구 목록 변경은 새로운 콘텐츠만 추가하고 이미 캐시된 항목을 방해하지 않습니다.

- **프리픽스에 로드된 도구:** 이에 대한 모든 변경은 캐시를 무효화합니다. 이는 도구 검색을 사용할 수 없거나 비활성화된 경우(예: Google Cloud의 Agent Platform 또는 사용자 정의 ANTHROPIC_BASE_URL 게이트웨이)에 발생합니다. 또한 `alwaysLoad` 로 표시된 서버 또는 도구, 그리고 임계값 기반 로딩으로 유지되는 정의에 대해서도 발생합니다.

도구가 프리픽스에 로드될 때 무효화의 가장 일반적인 원인은 **세션 중에 서버가 연결 또는 연결 해제**되는 것입니다. 이는 사용자의 조치 없이 발생할 수 있습니다: studio 서버의 프로세스가 종료되거나, HTTP 세션이 만료되거나, 서버가 일시적 오류 후 자동으로 재연결됩니다. 연결된 서버는 도구 목록을 변경하는 동적 도구 업데이트를 푸시할 수도 있습니다.

MCP 구성을 편집해도 캐시가 자동으로 변경되지 않습니다. **새로운 구성은 재시작 후에만 적용**되며, 이때 서버가 연결 또는 연결 해제됩니다.

캐시가 무효화되면서 어려움을 겪거나 당황했던 경험이 있는지?

플러그인 활성화 또는 비활성화

플러그인은 여러 구성 요소 유형을 번들로 제공하며, 변경 비용은 플러그인이 제공하는 구성 요소에 따라 달라집니다. Skills, commands, agents, hooks, LSP 서버, monitors, themes는 캐시를 무효화하지 않습니다: 이들이 요청에 추가하는 모든 것은 기존 대화 후에 추가되므로 다음 요청은 새로운 콘텐츠에 대해 비용을 지불하지만 여전히 그 이전의 모든 것을 캐시에서 읽습니다.

예외는 **MCP 서버**를 제공하는 플러그인입니다. **하나를 활성화 또는 비활성화하면 MCP 서버 연결 또는 연결 해제**와 동일한 규칙을 따릅니다: 서버의 도구가 연기될 때 캐시가 유지되고, 프리픽스에 로드될 때 다음 요청이 전체 대화를 다시 읽습니다.

플러그인 변경은 `/reload-plugins` 를 실행하거나 새 세션을 시작할 때 적용됩니다. 비용(추가된 공지 사항이든 전체 다시 읽기든)은 다시 로드 후 첫 턴에 표시되며, `/plugin install` , `/plugin enable` 또는 `/plugin disable` 을 실행할 때가 아닙니다. v2.1.163부터 다시 로드가 전체 다시 읽기를 트리거할 때 `/reload-plugins` 는 경고를 표시하고 다시 로드를 적용하지 않습니다. `--force` 를 전달하여 어차피 적용합니다.

세션 초반에 활성화한 플러그인을 비활성화하면 이전 요청 형태가 복원됩니다. 해당 프리픽스가 여전히 **캐시 수명** 내에 있으면 다음 요청이 다시 구축하는 대신 이전 캐시 항목을 읽습니다.

전체 도구 거부

`Bash` 또는 `WebFetch` 와 같은 단순 도구 이름을 **거부 규칙**으로 추가하면 해당 도구가 Claude의 컨텍스트에서 완전히 제거됩니다. 기본 제공 도구 정의는 시스템 프롬프트 계층에 로드되므로 이러한 규칙을 추가하거나 제거하면 세션 중에 캐시가 무효화됩니다. 변경 사항은 `/permissions` 를 통해 추가하는 설정 파일을 직접 편집 하든 다음 턴에 적용됩니다.

단순 도구 이름, 동등한 `Bash(*)` 형식, 또는 `"*"` 와 같은 도구 이름 글로브만 이 효과를 가집니다.

`"mcp_**"` 와 같이 MCP 도구만 일치하는 글로브는 해당 도구를 동일한 방식으로 제거하지만 일치하는 도구가

연기될 때 캐시를 그대로 유지합니다. 기본값이므로 연기된 정의는 캐시된 프리픽스에 없었습니다. `Bash(rm *)`와 같은 범위가 지정된 거부 규칙과 모든 허용 및 요청 규칙은 Claude가 보는 도구를 변경하지 않습니다. Claude Code는 Claude가 호출을 시도할 때 이를 확인하여 프리픽스를 그대로 유지합니다.

대화 압축

압축은 메시지 기록을 요약으로 바꿉니다. 설계상 이는 대화 계층을 무효화합니다. 다음 요청에는 이전 요청과 프리픽스를 공유하지 않는 새로운 더 짧은 기록이 있기 때문입니다. Claude Code는 시스템 프롬프트 계층을 재사용하고 디스크에서 프로젝트 컨텍스트를 다시 로드합니다. 이는 세션 시작 이후 CLAUDE.md와 메모리가 변경되지 않은 경우에만 캐시 히트됩니다.

요약을 생성하기 위해 Claude Code는 대화와 동일한 시스템 프롬프트, 도구 및 기록을 가진 일회성 요청을 보내고, 최종 사용자 메시지로 요약 지침을 추가합니다. 프리픽스를 공유하기 때문에 해당 요청은 전체 기록을 다시 처리하는 대신 기존 캐시를 읽습니다. 압축 시간의 대부분은 캐시 미스가 아닌 요약 생성에 소요됩니다. 뒤따르는 턴은 훨씬 더 짧은 요약에 대해서만 대화 캐시를 다시 구축하므로 압축 후 턴은 느린 부분이 아닙니다.

💡 압축은 더 이상 필요하지 않은 컨텍스트를 버릴 때 유리합니다. 오버헤드가 발생할 시기를 선택하려면 작업 간과 같은 자연스러운 휴식 시간에 `/compact`를 실행하고, 자동 압축이 작업 중에 트리거될 때까지 기다리지 마세요. 완전히 포기하고 싶은 경로를 따라가셨다면 압축이 새로운 경로를 구축하는 대신 이미 캐시된 프리픽스로 다시 자르는 `/rewind`를 사용하세요. **compact vs rewind**

Claude Code 업그레이드

새로운 Claude Code 버전은 일반적으로 시스템 프롬프트 또는 도구 정의를 업데이트하므로 업그레이드 후 첫 번째 요청은 캐시를 처음부터 다시 구축합니다. 자동 업데이트는 백그라운드에서 새 버전을 다운로드하지만 다음 시작 시에만 적용하므로 세션 중에 놀라운 일이 아닌 재시작 후 캐시되지 않은 첫 턴으로 표시됩니다.

`DISABLE_AUTOUPDATER=1`을 설정하여 업그레이드가 적용되는 시기를 제어합니다.

❗ 업그레이드 후 세션을 재개하면 캐시 히트 없이 전체 대화 기록을 다시 처리합니다. 기록이 이제 다른 시스템 프롬프트 뒤에 있기 때문입니다. 비용은 재개된 대화의 길이에 따라 확장되므로 긴 세션으로 돌아가는 첫 턴이 보내는 가장 비싼 요청일 수 있습니다.

캐시를 유지하는 작업

이러한 작업은 대화의 끝에 추가되거나 요청을 전혀 건드리지 않습니다. CLAUDE.md 편집 또는 출력 스타일 변경과 같은 일부는 설정 변경이 재시작을 기다리는 이유이기도 합니다.

- 저장소의 파일 편집
- 세션 중 CLAUDE.md 편집
- 출력 스타일 변경

- 권한 모드 변경
- 스킬 및 명령 호출
- /recap 실행
- 대화 되감기
- 서브에이전트 생성

저장소의 파일 편집

파일 콘텐츠는 Claude가 읽을 때만 컨텍스트에 들어가며, 읽기는 대화에 추가됩니다. Claude가 이전에 읽은 파일을 편집해도 기록의 이전 읽기를 소급하여 변경하지 않습니다. 대신 Claude Code는 파일이 변경되었음을 나타내는 `<system-reminder>` 를 추가하고, 필요한 경우 Claude가 다시 읽습니다.

세션 중 CLAUDE.md 편집

프로젝트 루트 및 사용자 수준 CLAUDE.md 파일은 세션 시작 시 한 번 읽혀지고 메모리에 보관됩니다. 세션 중에 편집해도 **캐시가 무효화되지 않지만 편집도 적용되지 않습니다.** Claude는 세션 시작 시 로드된 버전으로 계속 작동합니다. 새로운 콘텐츠는 다음 `/clear` , `/compact` 또는 재시작 시 로드됩니다.

하위 디렉토리의 중첩된 CLAUDE.md 파일과 paths: 프론트매터가 있는 규칙은 나중에 로드되며, Claude가 일치하는 파일을 처음 읽을 때 로드됩니다. 로드되기 전에 편집하면 적용됩니다. 로드된 후 콘텐츠는 대화 기록의 일부이므로 세션 중 편집은 소급하여 변경하지 않습니다.

출력 스타일 변경

출력 스타일은 Claude Code가 세션 시작 시 한 번 읽는 시스템 프롬프트의 일부입니다. `/config` 또는 `outputStyle` 설정을 통해 세션 중에 변경해도 캐시가 무효화되지 않지만 변경도 적용되지 않습니다. Claude는 세션 시작 시 로드된 스타일을 계속 사용합니다. 새로운 스타일은 다음 `/clear` 또는 재시작 시 로드됩니다.

권한 모드 변경

권한 모드 간 전환(예: 기본값에서 편집 수락으로)은 시스템 프롬프트 또는 도구 정의를 변경하지 않으므로 모드 변경은 캐시 안전입니다. 예외는 `opusplan` 모델 설정이 있는 Plan 모드입니다. 이는 Plan 모드에 들어가거나 나갈 때 모델을 Opus와 Sonnet 간에 전환합니다. 이는 모드 토크를 모델 전환으로 만듭니다.

스킬 및 명령 호출

스킬과 명령은 호출 지점에서 사용자 메시지로 지침을 주입합니다. 대화의 이전 내용은 변경되지 않습니다.

`/recap` 실행

`/recap` 은 **터미널에 표시할 요약**을 생성합니다. `/compact` 와 달리 메시지 기록을 바꾸는 대신 요약을 명령 출력으로 추가하므로 캐시된 프리픽스는 그대로 유지됩니다.

대화 되감기

`/rewind` 는 대화를 **이전 턴으로 자릅니다**. 남은 기록은 그 시점에서 캐시가 구축된 동일한 콘텐츠이고, 시스템 프롬프트 및 프로젝트 컨텍스트 계층은 변경되지 않으므로 다음 요청은 이전 캐시 항목에 히트합니다. 그 이후의 모든 턴은 해당 프리픽스를 통해 읽었으며, 원래 턴이 TTL보다 오래 전이었더라도 항목을 따뜻하게 유지했습니다.

대화과 함께 파일 체크포인트를 복원해도 캐시에 별도의 영향을 주지 않습니다. 파일 콘텐츠는 저장소의 파일 편집 과 동일하게 Claude가 읽을 때만 컨텍스트에 들어갑니다.

캐시 수명

캐시된 프리픽스는 비활성 기간 후에 만료됩니다. 캐시에 히트하는 각 요청은 타이머를 재설정하므로 계속 작업하는 한 캐시는 따뜻하게 유지됩니다. **충분히 긴 간격 후에 다음 요청은 전체 입력을 다시 계산하고 캐시를 다시 설정**합니다. 이것이 한동안 떨어진 후 돌아오는 첫 턴이 눈에 띄게 느릴 수 있는 이유입니다.

TTL(Time To Live)은 **캐시가 건디는 간격의 길이를 제어**합니다. API는 두 가지를 제공합니다: 5분 TTL과 더 긴 휴식을 통해 캐시를 따뜻하게 유지하지만 **캐시 쓰기를 더 높은 속도로 청구하는 1시간 TTL**. **Claude Code는 인증 방식에 따라 TTL을 선택하며, 환경 변수로 재정의할 수 있습니다.** [모 어떻게 한다는 이야기?](#)

Claude 구독에서

Claude 구독에서 Claude Code는 **자동으로 1시간** TTL을 요청합니다. 사용량은 토큰당 청구되지 않고 계획에 포함되므로 더 긴 TTL은 추가 비용이 없으며 캐시가 따뜻하게 유지되는 기간에만 영향을 줍니다.

계획의 사용량 한도를 초과했고 Claude Code가 사용 크레딧을 사용 중인 경우 해당 사용량에 대해 청구되므로 Claude Code는 자동으로 TTL을 5분으로 낮춥니다.

API 키 또는 타사 제공자에서

API 키, Amazon Bedrock, Google Cloud의 Agent Platform, Microsoft Foundry 또는 AWS의 Claude Platform에서 토큰당 요금을 지불하므로 TTL은 기본적으로 더 저렴한 **5분**으로 유지됩니다. **1시간 TTL**을 선택하려면 `ENABLE_PROMPT_CACHING_1H=1` 을 설정합니다. [이렇게 했을때 괜찮은지? 5분과 1시간은 간격이 너무 긴데.. 차이점](#)

Amazon Bedrock에서 프롬프트 캐싱 지원, 최소 캐시 가능 프리픽스 길이 및 1시간 TTL 가용성은 모두 모델에 따라 다릅니다. 캐시 토큰 수가 0으로 유지되면 Amazon Bedrock 설명서에서 지원되는 모델, 지역 및 제한을 확인하세요.

TTL 재정의 TTL을 직접 설정해서 사용하는 사람이 있는지?

`FORCE_PROMPT_CACHING_5M=1` 을 설정하여 인증에 관계없이 5분 TTL을 강제합니다. 이는 캐시 동작을 디버깅하거나, 두 TTL을 비교하거나, 관리 설정에 설정된 `ENABLE_PROMPT_CACHING_1H` 을 재정의할 때 유용합니다.

캐시 범위

Claude Code에서 캐시는 효과적으로 한 대의 머신과 디렉토리로 범위가 지정됩니다. 시스템 프롬프트는 작업 디렉토리, 플랫폼, 셀, OS 버전 및 자동 메모리 경로를 포함하므로 다른 디렉토리의 두 세션은 다른 프리픽스를 구축하고 서로의 캐시를 놓칩니다. 여기에는 동일한 저장소의 worktree가 포함됩니다. 각 worktree는 자체 작업 디렉토리를 가지기 때문입니다.

조금 더 자세한 동작방식을 알아보면 좋을듯
동일한 디렉토리에서 병렬로 실행하는 세션은 일치하는 프리픽스를 구축하고 서로의 캐시를 읽습니다. 순차 세션은 시작 시 git 상태 스냅샷이 일치할 때만 프리픽스를 공유합니다. 시스템 프롬프트도 분기 및 최근 커밋을 캡처하기 때문입니다.

기본 API 캐시는 더 광범위합니다. 캐시는 조직 간에 격리되며, 일부 제공자에서는 조직 내 워크스페이스 간에 격리됩니다. 이러한 경계 내에서 동일한 모델과 프리픽스를 가진 두 요청은 동일한 캐시를 읽습니다. 자동화된 프로세스의 플릿을 실행하는 Agent SDK 호출자의 경우 사용자 및 머신 간 프롬프트 캐싱 개선을 참조하여 시스템 프롬프트의 머신별 섹션을 억제하고 머신 간 캐시를 공유합니다.

캐시 성능 확인

캐시 성능은 API가 모든 응답에 보고하는 두 개의 토큰 수로 표시됩니다. 라이브로 감시하는 가장 직접적인 방법은 `current_usage` 객체를 읽는 상태 줄 스크립트입니다:

필드	의미
<code>cache_creation_input_tokens</code>	이 턴에서 캐시에 기록된 토큰, 캐시 쓰기 속도로 청구됨
<code>cache_read_input_tokens</code>	이 턴에서 캐시에서 제공된 토큰, 표준 입력 속도의 약 10%로 청구됨

높은 읽기 대 생성 비율은 캐싱이 잘 작동하고 있음을 의미합니다. 생성이 턴마다 높게 유지되면 프리픽스에서 원가 변경되고 있습니다. 캐시를 무효화하는 작업 섹션에서 일반적인 원인을 나열합니다.

조직 전체의 가시성을 위해 OpenTelemetry 내보내기는 사용자 및 세션당 캐시 읽기 및 생성 토큰을 보고합니다. 메트릭 및 이벤트 속성 참조는 사용량 모니터링을 참조하세요.

서브에이전트 및 캐시

서브에이전트는 부모와 별개의 자체 대화를 시작하며, 자체 시스템 프롬프트 및 도구 집합을 가집니다. 자체 캐시를 구축하며, 첫 호출 시 캐시 히트가 없고 자체 턴에 걸쳐 따듯해집니다. 서브에이전트는 자동 1시간 TTL이 주 대화에 적용되더라도 5분 TTL을 사용합니다.

부모의 캐시는 영향을 받지 않습니다. 부모 측에서 서브에이전트의 호출 및 결과는 대화에 추가되어 부모의 프리픽스를 그대로 유지합니다.

포크는 대조적으로 부모의 시스템 프롬프트, 도구 및 대화 기록을 정확히 상속하므로 첫 요청은 부모의 캐시를 읽습니다. 대화 압축에서 설명한 압축 요약 호출은 동일한 프리픽스 공유 접근 방식을 사용합니다.

프롬프트 캐싱 비활성화

캐싱을 비활성화하는 것은 특정 모델 또는 제공자로 캐싱 동작을 디버깅할 때 가끔 유용합니다. 그러면 다음 환경 변수 중 하나를 1로 설정합니다:

변수	효과
DISABLE_PROMPT_CACHING	모든 모델에 대해 비활성화
DISABLE_PROMPT_CACHING_HAIKU	Haiku만 비활성화
DISABLE_PROMPT_CACHING_SONNET	Sonnet만 비활성화
DISABLE_PROMPT_CACHING_OPUS	Opus만 비활성화
DISABLE_PROMPT_CACHING_FABLE	Fable만 비활성화

조직 전체에 캐싱 정책을 설정하려면 이 중 하나 또는 TTL 변수를 관리 설정의 env 블록에 넣습니다. 정상 사용의 경우 캐싱을 활성화된 상태로 유지합니다.

관련 리소스

- Claude Code 구축에서 배운 교훈: 프롬프트 캐싱이 모든 것입니다: Plan 모드, 연기된 도구 로딩 및 압축의 설계 근거
- 컨텍스트 윈도우 탐색: 컨텍스트에 로드되는 내용 및 시기
- 토큰 사용량 감소: 컨텍스트 크기 관리를 위한 캐싱 이상의 전략
- 비용 추적 및 감소: Agent SDK 호출자를 위한 캐시 토큰 추적 및 TTL 구성
- 프롬프트 캐싱: 기본 API 메커니즘, 중단점 및 가격 책정

이 페이지가 도움이 되었나요?

👍 예

👎 아니오

컨텍스트 윈도우 살펴보기

지침 및 메모리 저장